LALSIMULATION ISSUE REPORT

IMRPhenomXP OpenMP data race corrupts the MSA and PNR Euler angles

REPRODUCED WITH
lalsimulation 6.2.0

PRESENT ON
lalsuite master `ba91eb79ca`

FILES
`LALSimIMRPhenomX.c`
`LALSimIMRPhenomX_precession.c`

WORKAROUND
`OMP_NUM_THREADS=1`

Summary

`IMRPhenomXPGenerateFD` evaluates the waveform in a `#pragma omp parallel for` loop. In that loop, the MSA angles (`PhenomXPrecVersion` 220–224) and the PNR tuned angles store per-frequency intermediate values in the shared `IMRPhenomXPrecessionStruct *pPrec`. Each thread overwrites values that another thread has written but not yet read, so the Euler angles at some frequencies are computed from another frequency's spin roots or PNR angles.

The output then depends on `OMP_NUM_THREADS` and changes from run to run with the same thread count. With `OMP_NUM_THREADS=1` the output is correct and reproducible. Single-threaded runs are therefore correct and stable, which is probably why this has gone unnoticed for a long time.

This affects the **default** `IMRPhenomXP` configuration. `PhenomXPrecVersion` defaults to 300, which resolves to the MSA angles of 223, and the output is bit-identical to an explicit 223. Any build with OpenMP enabled is affected, e.g. conda-forge's, whose `liblalsimulation` links `libomp`. Users who leave `OMP_NUM_THREADS` unset get one thread per core, so the race is active.

IMRPhenomXPHM is **not** affected, because its loops have no OpenMP pragmas.

Affected configurations

These were measured with lalsimulation 6.2.0 (conda-forge, arm64 macOS) and `SimInspiralChooseFDWaveform`, using `IMRPhenomXP` with m1 = 30, m2 = 10 Msun, χ1 = (0.4, −0.3, 0.2), χ2 = (0.1, 0.5, −0.3), deltaF = 0.125 Hz and 20–1024 Hz (8193 bins). Each value is max |h(N threads) − h(1 thread)| / max |h(1 thread)| over both polarizations, for three separate runs.

CONFIGURATION	`OMP_NUM_THREADS=1`, REPEATED	`OMP_NUM_THREADS=8`, THREE RUNS
default (`PhenomXPrecVersion` 300 → MSA)	0	3.7e-2, 6.0e-2, 7.2e-2 RACES
`PhenomXPrecVersion` 223 (MSA)	0	3.9e-2 to 7.9e-2 (2, 4 and 8 threads) RACES
223, 150 + 100 Msun	0	2.8e-2, 9.8e-2, 8.3e-2 RACES
320 falling back to MSA (150 + 100 Msun)	0	1.4e-1, 1.6e-1, 2.1e-1 RACES
PNR tuned angles (`PhenomXPNRUseTunedAngles=1`)	0	1.2, 1.2, 1.0 RACES
`PhenomXPrecVersion` 102 (NNLO)	0	0 DETERMINISTIC
`PhenomXPrecVersion` 320 (SpinTaylor)	0	0 DETERMINISTIC
`IMRPhenomXPHM`, 223	0	0 DETERMINISTIC

For 223, between 88 and 1372 of the 8193 bins differ by more than 1e-10 relative, depending on the run. In the single-threaded runs, the 320 → MSA fallback is bit-identical to a direct 223 call.

The NNLO and SpinTaylor angles are deterministic because they don't write to `pPrec` inside the loop. NNLO angles are computed from read-only fields; SpinTaylor angles are precomputed into arrays and passed through `IMRPhenomXPTwistUp22_NumericalAngles`.

Reproducer

`scripts/reproduce_lal_openmp_race.py` (in the waveformrs repository) runs the waveform in fresh processes with different `OMP_NUM_THREADS`, because the variable is read when the OpenMP runtime starts:

PYTHON

```
CHILD = r"""
import sys
import numpy as np
import lal, lalsimulation as ls

version = int(sys.argv[1])
params = lal.CreateDict()
ls.SimInspiralWaveformParamsInsertPhenomXPrecVersion(params, version)
hp, hc = ls.SimInspiralChooseFDWaveform(
    30 * lal.MSUN_SI, 10 * lal.MSUN_SI, 0.4, -0.3, 0.2, 0.1, 0.5, -0.3,
    400e6 * lal.PC_SI, 0.7, 0.1, 0.0, 0.0, 0.0,
    0.125, 20.0, 1024.0, 20.0, params, ls.IMRPhenomXP,
)
np.save(sys.argv[2], np.stack([hp.data.data, hc.data.data]))
"""
# run CHILD with OMP_NUM_THREADS=1 and =8 and compare the saved arrays
```

Output, abridged:

OUTPUT

```
PhenomXPrecVersion 223: 8193 frequency bins
  OMP_NUM_THREADS=1 (repeat) max |dh|/max|h| = 0.00e+00   bins differing by >1e-10: 0
  OMP_NUM_THREADS=2          max |dh|/max|h| = 6.03e-02   bins differing by >1e-10: 500
  OMP_NUM_THREADS=4          max |dh|/max|h| = 7.85e-02   bins differing by >1e-10: 1372
  OMP_NUM_THREADS=8          max |dh|/max|h| = 3.91e-02   bins differing by >1e-10: 88
  OMP_NUM_THREADS=8          max |dh|/max|h| = 6.26e-02   bins differing by >1e-10: 1193
PhenomXPrecVersion 102: ... all 0
PhenomXPrecVersion 320: ... all 0
```

Root cause and lines to fix

Line numbers are given for lalsuite `master` at `ba91eb79ca` (lalsuite-v7.26.16-1575), with lalsimulation **v6.2.0** in brackets where they differ. Paths are relative to `lalsimulation/lib/`.

1. MSA angles write per-frequency state into `pPrec` (main bug)

The parallel loop is at `LALSimIMRPhenomX.c:2281` [2283]:

C

```
#pragma omp parallel for
for (UINT4 idx = 0; idx < freqs->length; idx++)
{
    ...
    IMRPhenomXPTwistUp22(Mf,hcoprec,pWF,pPrec,&hplus,&hcross);   /* :2401 [2403] */
```

For PrecVersion 220–224, `IMRPhenomXPTwistUp22` calls `IMRPhenomX_Return_phi_zeta_costhetaL_MSA(v,pWF,pPrec)` at `LALSimIMRPhenomX_precession.c:2040` [2009]. That function, defined at `:2229` [2198], computes the frequency-dependent roots of the S^2 cubic and stores them in the shared struct before reading them back:

C

```
/* LALSimIMRPhenomX_precession.c:2260-2273  [v6.2.0: 2229-2242] */
const vector vRoots    = IMRPhenomX_Return_Roots_MSA(L_norm,J_norm,pPrec);

pPrec->S32  = vRoots.x;                      /* written per frequency ... */
pPrec->Smi2 = vRoots.y;
pPrec->Spl2 = vRoots.z;

pPrec->Spl2mSmi2   = pPrec->Spl2 - pPrec->Smi2;
pPrec->Spl2pSmi2   = pPrec->Spl2 + pPrec->Smi2;
pPrec->Spl         = sqrt(pPrec->Spl2);
pPrec->Smi         = sqrt(pPrec->Smi2);

const double SNorm = IMRPhenomX_Return_SNorm_MSA(v,pPrec);   /* ... and read back */
pPrec->S_norm      = SNorm;
pPrec->S_norm_2    = SNorm * SNorm;

vector vMSA = {0.,0.,0.};
if(fabs(pPrec->Smi2 - pPrec->Spl2) > 1.e-5)                  /* read back */
{
  vMSA = IMRPhenomX_Return_MSA_Corrections_MSA(v, L_norm, J_norm, pPrec);
}
```

These functions, all called from this wrapper, read the fields back:

FUNCTION (`LALSimIMRPhenomX_precession.c`)	FIELDS READ
`IMRPhenomX_Return_SNorm_MSA` (:3239)	`S32` , `Smi2` , `Spl2`
`IMRPhenomX_Return_MSA_Corrections_MSA` (:3551)	`Spl2` , `Spl2mSmi2`
`IMRPhenomX_Return_Constants_c_MSA` (:3334), via the corrections	`Smi2` , `Spl2`
`IMRPhenomX_Return_Constants_d_MSA` (:3386), via the corrections	`Smi2` , `Spl` , `Spl2`
`IMRPhenomX_Return_Psi_dot_MSA` (:3431), via the corrections	`Spl2` , `S32`

Thread A can write its roots, then thread B overwrites them, then thread A evaluates `SNorm` and the MSA corrections with B's roots. α, ε and cos β at A's frequency are then wrong. This is a data race, so it is also undefined behaviour in C/OpenMP.

Fix. Keep the per-frequency roots local to the call. Nothing outside the wrapper reads these fields after `IMRPhenomX_Initialize_MSA_System` has used them. The other readers are at `:2578-2580` , `:2777-2782` and `:3005-3051` , all inside the initialisation, which sets the fields itself at `:2532-2547` . So making the values local should leave the single-threaded output bit-identical. There are two options.

- **Minimal patch:** work on a private copy of the struct inside the wrapper. The shallow copy shares the pointer members read-only; it costs a `memcpy` of the struct per call, small next to the elliptic-function evaluation.

  PATCH

```
 vector IMRPhenomX_Return_phi_zeta_costhetaL_MSA(
   const double v,                   /**< Velocity                         */
   IMRPhenomXWaveformStruct *pWF,    /**< IMRPhenomX waveform struct       */
-  IMRPhenomXPrecessionStruct *pPrec /**< IMRPhenomX precession struct     */
+  const IMRPhenomXPrecessionStruct *pPrecShared /**< IMRPhenomX precession struct */
 )
 {
+  /* The S^2 roots and |S| depend on v: keep them in a private copy so this
+     function can be called from the OpenMP loop in IMRPhenomXPGenerateFD. */
+  IMRPhenomXPrecessionStruct precLocal = *pPrecShared;
+  IMRPhenomXPrecessionStruct *pPrec = &precLocal;
   vector vout = {0.,0.,0.};
```

  Update the prototype in `LALSimIMRPhenomX_precession.h` to match. Callers keep passing `pPrec` , since a non-const to const pointer conversion is implicit.

- **Cleaner patch:** add a small per-frequency struct, e.g. `IMRPhenomXMSAState { S32, Smi2, Spl2, Spl2mSmi2, Spl2pSmi2, Spl, Smi, S_norm, S_norm_2 }` . Fill it in the wrapper and pass it, together with

`const IMRPhenomXPrecessionStruct *`, to `IMRPhenomX_Return_SNorm_MSA`, `IMRPhenomX_Return_MSA_Corrections_MSA`, `IMRPhenomX_Return_Constants_c_MSA`, `IMRPhenomX_Return_Constants_d_MSA` and `IMRPhenomX_Return_Psi_dot_MSA`. The initialisation at `:2532-2547` can fill the same struct for v_0. This also lets the compiler enforce the read-only use of `pPrec` in the per-frequency code.

2. PNR angles are passed through `pPrec`

In the same loop (`LALSimIMRPhenomX.c:2390-2395` [2392-2397]):

```c
if(PNRUseTunedAngles) /* Look for PNR flag */
{
  pPrec->alphaPNR = alphaPNR->data[idx];     /* :2392 [2394] */
  pPrec->betaPNR = betaPNR->data[idx];       /* :2393 [2395] */
  pPrec->gammaPNR = gammaPNR->data[idx];     /* :2394 [2396] */
}
```

These are read inside `IMRPhenomXPTwistUp22` at `LALSimIMRPhenomX_precession.c:2002-2004` [1971-1973]:

```c
alpha       = pPrec->alphaPNR - pPrec->alpha_offset;
epsilon     = -1.0 * pPrec->gammaPNR - pPrec->epsilon_offset;
cos_beta    = cos(pPrec->betaPNR);
```

Another thread's frequency can be used here, so the errors are O(1).

Fix. Pass the angles explicitly instead of through the struct. One way is to split `IMRPhenomXPTwistUp22` into an angle step and a twist step. The angle step computes α, ε and cos β (NNLO, MSA or PNR). The twist step takes `(hAS, alpha, epsilon, cos_beta, pPrec)` and does the Wigner-d sum, with `pPrec` only read. The loop then passes `alphaPNR->data[idx] - pPrec->alpha_offset`, `-gammaPNR->data[idx] - pPrec->epsilon_offset` and `cos(betaPNR->data[idx])` directly. The existing `IMRPhenomXPTwistUp22_NumericalAngles` can't be reused as-is, because it applies the SpinTaylor convention `epsilon = -(gamma - gamma_ref) + epsilon0`.

3. Shared scalars written inside the parallel loop (secondary)

These don't change results in the cases measured above, but they are also data races:

- `phi_antiSym` and `amp_antiSym` are declared before the loop at `LALSimIMRPhenomX.c:2254-2255` [2256-2257] and assigned in the loop body at `:2412`, `:2416` and `:2421`. With `PhenomXAntisymmetricWaveform`

enabled they are shared between threads. **Fix:** declare them inside the loop body. The antisymmetric branch also calls `IMRPhenomXPTwistUp22` with the PNR angles at `:2424` [2426], so it is affected by item 2 as well.

- `initial_status` is declared outside both parallel loops, at `:783` [785] for `IMRPhenomXASGenerateFD` and `:2205` [2207] for `IMRPhenomXPGenerateFD`. It is assigned in the loop bodies at `:809` and `:2297`. `status` is then assigned from several threads at `:812` and `:2300`. In practice every thread writes `XLAL_SUCCESS`, but a failure in one thread can be overwritten, and concurrent writes are undefined behaviour. **Fix:** declare `initial_status` inside the loop. Record failures with `#pragma omp atomic write` on `status`, or with a `reduction(max:failed)` flag checked after the loop.

Suggested regression test

Generate `IMRPhenomXP` with PrecVersion 223, and with `PhenomXPNRUseTunedAngles=1`, once with `omp_set_num_threads(1)` and once with several threads. Assert that the outputs are bit-identical. A C test in `lalsimulation/test/` can call `omp_set_num_threads` directly. In Python, `OMP_NUM_THREADS` must be set before the process starts, as in the reproducer. Running the existing IMRPhenomXP tests under ThreadSanitizer (`-fsanitize=thread` with an OpenMP runtime built for TSan) should flag items 1–3 directly.

Workaround until fixed

Set `OMP_NUM_THREADS=1`, or use `PhenomXPrecVersion` 102 or 320. Note that 320 silently falls back to MSA, and so to the race, for many heavy systems. In our random draws, 16 of 200 binaries above about 150 Msun fell back. `IMRPhenomXPHM` is unaffected.

Notes

- This was found while porting the MSA angles to the Rust crate waveformrs, where the single-threaded LAL output is reproduced to ~1e-12.
- An earlier comparison in that work found that LAL's 320 → MSA fallback differed from a direct 223 call by 7e-4 to 3e-3. With `OMP_NUM_THREADS=1` the two agree bit for bit, so that difference was most likely this race.